

Day 0 - Before You Write Any Code: The Absolute Basics

Day 0 — Before You Write Any Code: The Absolute Basics

This day exists so that every lesson after it can assume you already know what HTML and CSS actually are, how a browser turns them into the page you see on screen, and how to peek "under the hood" of any web page using your browser's built-in developer tools. If you've never built a web page before, read this first — it will make Day 1 dramatically less overwhelming. If any of this is already familiar, skim it and move on.

What actually happens when you visit a web page?

When you open a web page — whether it's a file sitting on your own computer or a real website out on the internet — your **browser** (Chrome, Firefox, Edge, Safari) downloads (or reads, for a local file) a text file written in **HTML**, and then does an enormous amount of work to turn that plain text into the colors, boxes, text, and images you actually see on screen. That whole process is called **rendering**.

Two separate languages work together to make this happen, and today's lesson (and this entire two-week track) is about both of them:

- **HTML** (HyperText Markup Language) describes the *structure and content* of a page — this is a heading, this is a paragraph, this is a picture, this is a link to another page. HTML says **what things are**, not what they look like.
- **CSS** (Cascading Style Sheets) describes the *appearance* of that structure — what color the heading is, how much space surrounds the paragraph, whether the picture is large or small, where things are positioned on the page. CSS says **how things look**.

A useful, genuinely accurate analogy: HTML is like the wooden frame and rooms of a house (this is a bedroom, this is a kitchen, this is the front door) — the structure exists and works correctly even if it's completely unpainted and undecorated. CSS is the paint, furniture, and interior design — it makes the exact same structural rooms look completely different depending on the choices made, without changing what each room fundamentally **is**.

You'll also frequently hear a third language mentioned alongside these two: **JavaScript**, which adds **behavior** — things that happen in response to clicks, typing, or time passing. This particular course only covers HTML and CSS; if you're pairing it with the separate JavaScript/TypeScript track in this same **learning-code** folder, that's where behavior is covered. For these two weeks, you're focused entirely on structure (HTML) and appearance (CSS) — genuinely enough, on its own, to build and style complete, real web pages.

What is a "tag," an "element," and an "attribute"?

These three words get used constantly from Day 1 onward, so let's nail down precisely what each means using one concrete example:

```
<a href="https://example.com">Click here</a>
```

- A **tag** is one specific piece of markup, written between angle brackets: `<a>` is the **opening tag**, and `</a>` (with the forward slash) is the **closing tag**. The name inside the brackets (`a`, here, which stands for "anchor" and is how you make a hyperlink) tells the browser what **kind** of thing this is.
- An **element** is the opening tag, everything between it and its matching closing tag, and the closing tag itself, all together, treated as one unit: the whole thing, `<a href="https://example.com">Click here</a>`, is "an `a` element" (informally, people often just say "a link element" or even just "a link").
- An **attribute** is extra information attached to the **opening** tag, written as `name="value"`: `href="https://example.com"` is an attribute named `href` (short for "hypertext reference"), telling the browser exactly which page this link should go to. An element can have zero, one, or several attributes.

Some elements don't wrap around any content and therefore have no separate closing tag at all — these are called **self-closing** (or "void") elements, and you'll meet a few of these very soon (e.g., `<img>` for an image, `<br>` for a line break).

The minimal shape of an HTML document

Every HTML file follows this same basic skeleton, which you'll type out (or let your editor auto-generate) at the top of essentially every page you ever create:

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

Reading through this line by line, since you'll want to fully understand every part before Day 1 adds more elements on top of it:

- `<!DOCTYPE html>` isn't really a tag in the same sense as the others — it's a required, one-line declaration telling the browser "interpret everything that follows using the modern, standard rules for HTML." Without it, some older browsers can fall back into inconsistent, legacy rendering behavior — always include it, as the very first line of every HTML file, with nothing before it.
- `<html>...</html>` wraps around the **entire** rest of the document — everything you write lives inside this one element.
- `<head>...</head>` contains information **about** the page that isn't itself displayed as visible content on the page — things like the page's title (shown in the browser tab), links to CSS files (Day 3), and other metadata.
- `<title>My First Page</title>`, inside the `<head>`, sets the text shown in the browser's tab or window title bar — notice this text is never shown **in** the page itself, only in the browser's own interface around it.
- `<body>...</body>` contains everything that actually gets displayed as visible content on the page — headings, paragraphs, images, links, all of it.
- `<h1>Hello, World!</h1>` is a top-level heading (you'll meet the full range of heading levels, `h1` through `h6`, on Day 1).

- `<p>This is a paragraph.</p>` is an ordinary paragraph of text.

Notice the **indentation** (the consistent spacing before `<title>`, `<h1>`, and `<p>`) — unlike Python, HTML's indentation has no effect whatsoever on how the browser interprets the file; it's purely there to help *you*, the human reading the code, visually see which elements are nested inside which other elements. Get in the habit of indenting consistently anyway — your code editor will typically do this for you automatically, and it makes deeply nested pages vastly easier to read later.

Comments — notes to humans that the browser ignores

HTML comments are written as `<!-- like this -->` — anything between those markers is completely ignored by the browser and never shown on the page; it exists purely as a note for whoever reads the source code later (including future-you):

```
<!-- This section is the site's main navigation bar -->
<nav>...</nav>
```

CSS has its own, different comment syntax, which you'll meet properly on Day 3: `/* like this */`.

How to actually open and view an HTML file

Save a file named `hello.html` somewhere on your computer with the skeleton shown above. To view it, you don't need any special program or "running" step the way Python or JavaScript needs an interpreter — you simply open the file directly in your web browser. On Windows, you can typically right-click the file, choose "Open with," and pick your browser — or simply double-click it if your browser is already set as the default program for `.html` files. Once open, you're looking at your browser's **rendering** of that file — the actual visual result of everything you wrote.

Whenever you change the file and want to see the update, you must save the file in your editor, then manually reload the page in your browser (the reload/refresh button, or Ctrl+R / F5) — the browser has no way of knowing you changed the file until you explicitly ask it to reload. This "edit, save, reload, look" cycle is the exact equivalent of Python's "write, run, read output" cycle from that track's Day 0, and you'll repeat it constantly throughout this course.

Browser DevTools — looking under the hood of any web page

Every modern browser has a built-in set of tools, collectively called **DevTools** ("Developer Tools"), for inspecting and even temporarily modifying any page you have open — including pages you didn't write yourself. This is one of the single most useful habits to build starting today, since it lets you directly connect a piece of HTML/CSS you wrote to exactly how the browser actually rendered it.

To open DevTools: right-click anywhere on a page and choose **"Inspect"** (sometimes labeled "Inspect Element"), or press **F12**, or **Ctrl+Shift+I** on Windows. This opens a panel (usually docked to the side or bottom of the browser window) with several tabs — the two you'll use constantly are:

- **Elements** (sometimes called "Inspector" in Firefox) — shows you the actual HTML structure the browser is currently using to render the page, and lets you click on any element to see exactly which CSS rules are

being applied to it, where those rules came from, and even temporarily edit them live to experiment (any changes you make here are temporary and disappear the moment you reload the page — this is a genuinely safe way to experiment without any risk of breaking your actual files).

- **Console** — shows error messages and lets you run small snippets of JavaScript directly against the current page. You won't use this heavily in an HTML/CSS-only course, but it's worth knowing it's there, since browsers will sometimes print warnings here about things like a missing image file or an invalid attribute.

A genuinely useful early habit: whenever a page you're building doesn't look the way you expected, open DevTools, click the **Elements** tab, and click directly on the misbehaving part of the page (there's usually a small cursor/pointer icon in the DevTools toolbar for exactly this — "select an element on the page"). This instantly shows you the exact HTML for that piece, and every single CSS rule currently affecting it — which is, in practice, how real web developers debug almost everything, every single day, rather than guessing.

Exercises

Open `starter.html` in this folder, then open it in your browser to see what it currently looks like. Follow the `<!-- TODO -->` comments inside it, editing the file in your code editor, saving, and reloading your browser after each change to see the result. When you're done, compare against `CHECKLIST.md` and, if you'd like, `solution.html`.

Day 1 - HTML Basics: Document Structure, Common Elements, Attributes

Day 1 — HTML Basics: Document Structure, Common Elements, Attributes

If you haven't read [day00-primer/lesson.md](#) yet, read that first — it covers what a tag, element, and attribute are, and the minimal shape of an HTML document, which everything below assumes you already know.

Objectives

- Get comfortable with the full basic document skeleton and why each part exists
- Meet the handful of elements you'll use in nearly every single page you ever build: headings, paragraphs, links, images, and lists
- Understand attributes in more depth, including ones that matter for accessibility (a theme you'll return to repeatedly)
- Understand *why* choosing the semantically correct element matters, not just picking whichever one happens to look right

Headings — `` through ``

HTML gives you six levels of heading, from most important (`<h1>`) to least important (`<h6>`):

```
<h1>Page Title</h1>
<h2>A Major Section</h2>
<h3>A Subsection</h3>
```

By default, browsers render `<h1>` the largest and boldest, shrinking progressively down to `<h6>`. It's tempting, as a beginner, to just pick whichever heading level happens to produce the font size you want visually — **resist this**. Heading levels are meant to describe the actual *outline* of your document's content, similar to how a book has chapters, then sections, then subsections — a screen reader (software that reads a page aloud for a visually impaired user) and search engines both rely on this structure to understand your page, completely independent of how large the text visually appears. If you want a heading to merely *look* smaller without changing its structural importance, you'll learn the correct tool for that on Day 3 (CSS) — don't misuse heading levels to fake a visual effect.

Use exactly one `<h1>` per page, representing the single main topic of that page, and nest the rest logically beneath it (don't jump from `<h1>` straight to `<h4>`, skipping levels).

Paragraphs — ` `

```
<p>This is a paragraph of text. It can contain several sentences,
and the browser will automatically wrap the lines to fit the available width.</p>
```

Notice something important: the line breaks *inside* your HTML source code (where you happened to press Enter while typing) have **no effect** on how the paragraph is displayed — the browser collapses any amount of whitespace (spaces, tabs, line breaks) in your source into a single space when rendering text. The paragraph will visually wrap based on the width of its container on screen, not based on where you happened to break lines in your source file. This surprises many beginners the first time they see it — go ahead and try adding extra line breaks and spaces inside a `<p>` in today's exercise, reload, and confirm the rendered result doesn't change.

Links — `<a>`

You already saw this on Day 0: `<a>` (short for "anchor") creates a hyperlink.

```
<a href="https://example.com">Visit Example</a>
<a href="about.html">About This Site</a>
<a href="#section2">Jump to Section 2</a>
<a href="mailto:someone@example.com">Email Us</a>
```

The `href` attribute (short for "hypertext reference") is what actually specifies the destination — the visible, clickable text between `<a>` and `</a>` can be completely different from the destination itself. A few different kinds of destinations worth knowing:

- A full web address (`https://example.com`) — an **absolute** link, pointing to a location anywhere on the internet.
- A filename or path relative to the current page (`about.html`) — a **relative** link, pointing to another page in the same project, without needing the full address.
- A `#` followed by an element's `id` attribute (explained below) — jumps to a specific spot *within the same page*, rather than loading a different page at all.
- `mailto:` — opens the visitor's email program with a new message addressed to that address, rather than navigating anywhere.

Images — `<img>`

```

```

`<img>` is a **self-closing** element (recall from Day 0 — no separate closing tag, since it doesn't wrap around any content). Two attributes matter enormously:

- `src` (source) — the file path or web address of the actual image to display.
- `alt` (alternative text) — a plain-text description of the image's content, displayed if the image fails to load, and — much more importantly — read aloud by screen readers for visually impaired users, since they have no way to actually see the image itself. **Always write a meaningful `alt`, describing what the image actually shows** — never leave it out, and never just repeat the filename. If an image is purely decorative and conveys no real information (a background flourish, say), the correct practice is `alt=""` (present, but deliberately empty) — this explicitly tells screen readers "skip this, there's nothing meaningful to describe," which is different, and better, than simply omitting `alt` entirely (which typically causes screen readers to read out the entire file name instead, which is much less useful).

Lists — ` `, ` `, and ` - `

Two kinds of lists, and one shared building block:

```
<ul>
  <li>Milk</li>
  <li>Eggs</li>
  <li>Bread</li>
</ul>

<ol>
  <li>Preheat the oven</li>
  <li>Mix the ingredients</li>
  <li>Bake for 20 minutes</li>
</ol>
```

`<ul>` is an **unordered list** (bullet points — use it when the sequence of items genuinely doesn't matter, like a shopping list). `<ol>` is an **ordered list** (automatically numbered — use it when order genuinely matters, like steps in a recipe). Every individual item, in either kind of list, is an `<li>` ("list item") — and every `<li>` must live directly inside a `<ul>` or `<ol>`, never on its own.

Attributes, in more depth

You met the general idea of an attribute on Day 0 (`name="value"`, written inside an opening tag). A few attributes are so common they're worth calling out specifically, since you'll use them across many different elements:

- `id` — gives one specific element a unique name, which must not be repeated anywhere else on the same page. Useful for linking to a specific spot (`<a href="#section2">` jumping to `<h2 id="section2">`) and, as you'll see from Day 3 onward, for targeting one specific element with CSS.
- `class` — gives an element (or, unlike `id`, potentially *many* elements) a label you can use to apply the same styling or behavior to a whole group at once. You'll use `class` constantly starting Day 3.
- `title` — adds a small tooltip that appears when a visitor hovers their mouse over the element.

```
<p id="intro" class="highlight" title="This is the introduction">Welcome to my site!</p>
```

Why "semantic" elements matter — a preview of Day 2

You'll notice, starting tomorrow, that HTML offers many elements beyond generic containers — things like `<nav>`, `<header>`, and `<article>` — whose entire purpose is to describe *what a piece of content actually is*, rather than just being an anonymous box you style however you like. The underlying principle, worth internalizing today even before you meet those specific elements tomorrow: **choose the HTML element that correctly describes what the content actually IS, not the one that happens to look right by default.** A screen reader, a search engine, and another developer reading your code later all rely on this correctness — CSS (starting Day 3) is the tool for controlling how something *looks*; HTML's job is to correctly describe what something *is*, first.

Exercises

Open `starter.html`, follow the `<!-- TODO -->` markers, and check your work by reloading the file in your browser and comparing against `CHECKLIST.md`.

Day 2 - HTML Forms, Tables, and Semantic HTML5

Day 2 — HTML Forms, Tables, and Semantic HTML5

Objectives

- Build a real HTML form with several input types, and understand the connection between a form and the data it would send somewhere
- Understand why `<label>` matters for accessibility, not just appearance
- Build a table for genuinely tabular data, and know when NOT to reach for one
- Meet HTML5's semantic layout elements and understand why they exist instead of just using `<div>` everywhere

Forms — collecting input from a visitor

A **form** is how a web page collects information from someone visiting it — a login screen, a search box, a contact form, a checkout page. The wrapping element is `<form>`:

```
<form action="/submit" method="post">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name">

  <button type="submit">Submit</button>
</form>
```

`action` says where the form's data should be sent when submitted (a web address on a server — you won't have a real server to send this to in this HTML/CSS-only course, so don't worry about `action` actually working yet; it becomes relevant once you're pairing this with a backend, which is outside this track's scope). `method` says how the data is sent ("`get`" or "`post`" — a distinction that matters once you're actually building a server to receive it, again outside today's scope). For today, you're focused on correctly *building* the form's structure — connecting it to a real server is a separate skill for later.

`<label>` and `for`/`id` — a genuinely important accessibility connection

```
<label for="email">Email address:</label>
<input type="email" id="email" name="email">
```

The `for` attribute on `<label>` must exactly match the `id` attribute on the input it describes. This creates a real, functional connection — not just a visual one — between the label text and its input: clicking the label text itself moves focus into (or, for a checkbox, toggles) the associated input, and screen readers announce the label's text whenever that input receives focus. **Never skip `<label>`, and never rely on placeholder text alone as a substitute** — placeholder text (shown below) disappears the instant someone starts typing, and isn't reliably announced by every screen reader the same way a proper `<label>` is.

Common `<input>` types

```








```

`type` changes both how the input is displayed (a password field hides typed characters; a number field often shows small up/down arrows) and, on many modern browsers and mobile devices, what kind of keyboard or built-in validation is offered (an `email` field can show an @-friendly keyboard on a phone, and browsers can reject obviously malformed input automatically — much more on this validation behavior on Day 12). Radio buttons that should behave as one connected group (only one selectable at a time) must all share the exact same `name` attribute, as shown above with `"size"` — this is how the browser knows they belong together and enforces "only one selected."

Other common form elements:

```

<textarea name="message" rows="4" cols="40">Default text here</textarea>

<select name="country">
  <option value="us">United States</option>
  <option value="ca">Canada</option>
  <option value="uk">United Kingdom</option>
</select>

<button type="submit">Send Message</button>

```

`<textarea>` is for multi-line free text (a comment box, a message). `<select>` with nested `<option>` elements creates a dropdown menu. `<button type="submit">` (or `<input type="submit">`) submits the form when clicked — note the difference between `type="submit"` (submits the form) and `type="button"` (does nothing on its own; used when you'll later attach custom JavaScript behavior to it, outside this course's scope).

Tables — for genuinely tabular data

```

<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Age</th>
      <th>City</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Ana</td>
      <td>30</td>
      <td>Boston</td>
    </tr>
    <tr>
      <td>Bo</td>
      <td>25</td>
      <td>Chicago</td>
    </tr>
  </tbody>
</table>

```

```

        </tr>
      </tbody>
    </table>

```

`<table>` wraps the whole table. `<thead>` groups the header row(s); `<tbody>` groups the actual data rows (both are optional but good practice, and genuinely useful once you meet CSS styling and want to target headers differently from data rows). `<tr>` is one table row ("table row"). Inside a row, `<th>` ("table header") marks a header cell (browsers bold and center it by default), while `<td>` ("table data") marks an ordinary data cell.

A genuinely important rule, worth stating plainly: only use `<table>` for data that is actually tabular — rows and columns of related data, like a spreadsheet, a price list, or a schedule. **Do not use a `<table>` purely to arrange unrelated content visually on the page** (like a page layout with a "sidebar column" and a "main content column") — this was common practice in the early days of the web, is now considered outdated and actively harmful to accessibility (screen readers announce table structure — row and column counts, headers — which is confusing and meaningless when the "table" was never really tabular data at all), and has been fully superseded by CSS layout tools you'll learn Day 5 (Flexbox) and Day 6 (Grid), which are the correct tools for arranging a page's visual layout.

Semantic HTML5 — elements that describe a page's actual layout regions

You've been using `<div>` and `<span>` implicitly so far only through prose descriptions — these are **generic, meaningless containers**: a `<div>` is a generic block-level box, and a `<span>` is a generic inline box, and neither says anything at all about what the content inside actually *is*. HTML5 introduced a set of elements that describe common, recognizable *regions* of a typical page layout, carrying real semantic meaning:

```

<body>
  <header>
    <h1>My Site</h1>
    <nav>
      <a href="/">Home</a>
      <a href="/about">About</a>
      <a href="/contact">Contact</a>
    </nav>
  </header>

  <main>
    <article>
      <h2>My First Blog Post</h2>
      <p>Content of the post goes here...</p>
    </article>

    <aside>
      <h3>Related Links</h3>
      <ul><li><a href="#">Another post</a></li></ul>
    </aside>
  </main>

  <footer>
    <p>&copy; 2026 My Site</p>
  </footer>
</body>

```

- `<header>` — introductory content for the page or a section (often contains a logo/title and navigation, but can appear more than once, e.g., inside an `<article>` for that article's own header).
- `<nav>` — a block of primary navigation links.
- `<main>` — the single, primary, unique content of the page (use exactly one per page).
- `<article>` — a self-contained, independently distributable piece of content — a blog post, a news story, a forum post — content that would still make complete sense on its own if pulled out and placed somewhere else entirely.
- `<section>` — a thematic grouping of content, generally with its own heading, when there isn't a more specific element (like `<article>`) that fits better.
- `<aside>` — content tangentially related to the surrounding content, like a sidebar, a pull-quote, or related links.
- `<footer>` — closing content for the page or a section (copyright notices, contact info, secondary links).

Why bother with these instead of just using `<div>` for everything, especially since, by default, many of these look identical to a `<div>` until you apply CSS? The exact same reasoning from Day 1's heading discussion applies here, at the scale of a whole page: screen readers can let a visitor jump directly to "the navigation" or "the main content" without listening to the entire page read aloud first; search engines use this structure to better understand which part of your page is the actual content versus boilerplate surrounding it; and another developer (including future-you) can understand your page's layout at a glance from the tag names alone, without needing to guess what a generically-named `<div class="thing1">` was supposed to represent.

A practical rule of thumb: reach for the semantic element whenever one genuinely describes the region you're building (header, nav, main, article, section, aside, footer). Fall back to a plain `<div>` only when you need a container purely for layout or styling purposes, with no real semantic meaning of its own — you'll use both throughout the rest of this course, and knowing which to reach for is itself a skill you're building starting today.

Exercises

Open `starter.html`, follow the `<!-- TODO -->` markers, and check your work against `CHECKLIST.md`.

Day 3 - CSS Basics: Selectors, the Cascade, Specificity, the Box Model

Day 3 — CSS Basics: Selectors, the Cascade, Specificity, the Box Model

Objectives

- Understand what a stylesheet is and the three ways to apply CSS to a page, and why one is strongly preferred
- Write selectors that target exactly the elements you want to style
- Understand the cascade and specificity — how the browser decides which rule wins when several could apply to the same element
- Understand the box model in real depth — arguably the single most important idea in all of CSS

What is CSS, mechanically?

CSS stands for Cascading Style Sheets. A **rule** consists of a **selector** (which elements should this apply to?) followed by a block of **declarations** in curly braces (what should change about them?):

```
p {  
  color: blue;  
  font-size: 18px;  
}
```

Read this as: "select every `<p>` element, and set its text color to blue and its font size to 18 pixels." Each individual line inside the braces — `color: blue;` — is one **declaration**, made of a **property** (`color`) and a **value** (`blue`), separated by a colon and ended with a semicolon.

Three ways to apply CSS — and why external is the one you should actually use

1. Inline — directly on one specific element, via a `style` attribute:

```
<p style="color: blue; font-size: 18px;">Some text</p>
```

2. Internal — inside a `<style>` block in the page's `<head>`:

```
<head>  
  <style>  
    p {  
      color: blue;  
    }  
  </style>  
</head>
```

3. External — in a completely separate `.css` file, linked from the HTML's `<head>`:

```

<head>
  <link rel="stylesheet" href="styles.css">
</head>

/* styles.css */
p {
  color: blue;
}

```

Always prefer external stylesheets for any real project, and that's what you'll do starting today's exercise. Why: it cleanly separates structure (HTML) from appearance (CSS), exactly matching the "wooden frame vs. paint and furniture" distinction from Day 0; it lets you apply the *exact same* styling to every single page of a multi-page site by linking the same one `styles.css` file from each page, instead of retyping styles everywhere; and it makes your HTML dramatically easier to read, since it isn't cluttered with styling details mixed into the content. Inline styles are occasionally useful for a genuinely one-off, single-element tweak, but relying on them as your main approach is considered poor practice in any real project — you'll be marked down for it, informally, by any reviewer at a job.

CSS comments look like this: `/* a comment */` — different syntax from HTML's `<!-- -->`, so don't mix them up.

Selectors — precisely choosing what to style

```

p { }                /* every <p> element */
.highlight { }       /* every element with class="highlight" (note the leading DOT) */
#main-title { }      /* the ONE element with id="main-title" (note the leading HASH/pound sign)
nav a { }            /* every <a> that is anywhere INSIDE a <nav> -- a "descendant" selector
p.intro { }          /* every <p> that ALSO has class="intro" -- no space, combining two
h1, h2, h3 { }       /* every h1 AND every h2 AND every h3 -- a comma separates a shared

```

Recall from Day 1: `class` can label many elements at once (so `.highlight` might match several elements across the page), while `id` must be unique to exactly one element per page (so `#main-title` matches, at most, exactly one). As a strong rule of thumb for the next two weeks: **reach for classes to style groups of similar elements** (this is what you'll do the vast majority of the time), **and reserve id selectors mainly for the rare cases where you genuinely need to target one single, unique element** (or, more often, for `href="#some-id"` in-page jump links from Day 1, and for JavaScript to hook into, which is outside this course's scope).

The cascade and specificity — how the browser resolves conflicts

The "C" in CSS stands for **Cascading** — meaning multiple rules can apply to the very same element simultaneously, and CSS needs a consistent, predictable set of rules for deciding which one actually wins when they conflict. This is one of the ideas beginners find most confusing at first, so let's build it up carefully.

Consider:

```

p { color: black; }
.warning { color: red; }
#special-message { color: green; }

<p id="special-message" class="warning">What color is this text?</p>

```

All three rules technically match this one `<p>` — so which color wins? CSS resolves this using **specificity**: a scoring system where more specific selectors beat more general ones, regardless of the order they're written in. The scoring, roughly, from lowest to highest specificity:

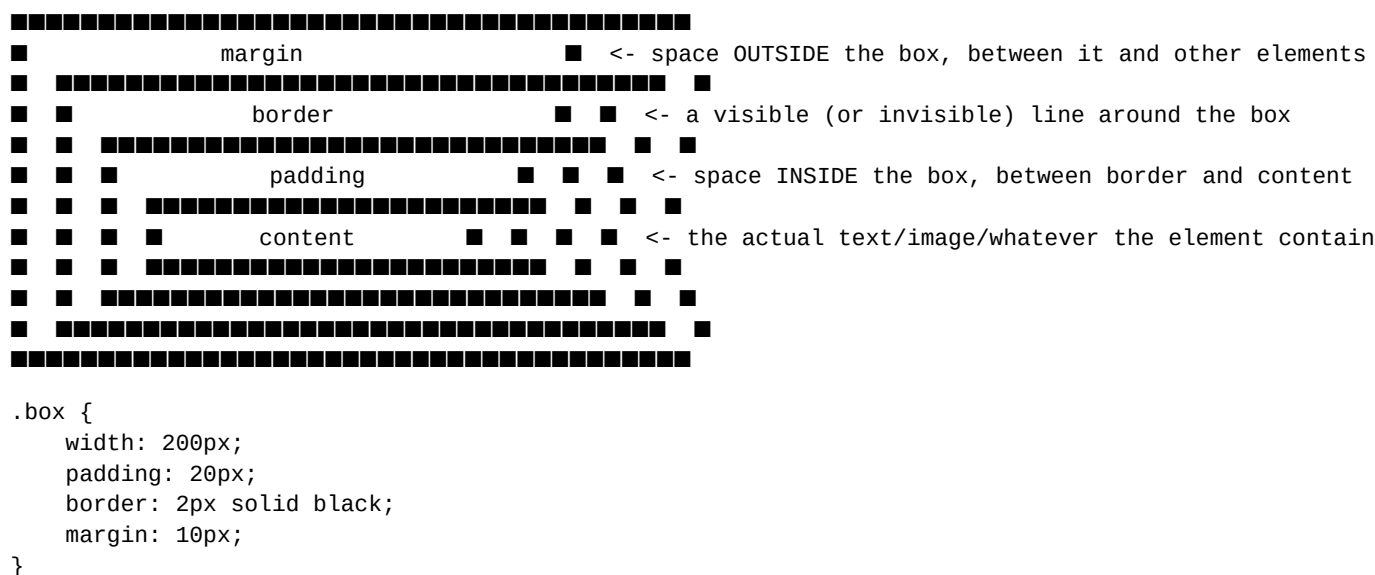
- 1 Element selectors (`p`, `div`, `a`) — the least specific.
- 2 Class selectors (`.warning`), and a couple of similar selector types you'll meet later (attribute selectors, pseudo-classes — Day 9).
- 3 ID selectors (`#special-message`) — more specific than any number of classes.
- 4 Inline `style="..."` attributes — beats essentially everything above.
- 5 `!important` appended to a declaration — an escape hatch that overrides normal specificity entirely; **avoid this in real code** — it's a sign of a specificity problem you should actually fix (often by simplifying your selectors, a topic you'll dig into on Day 11), not paper over, since once you start using `!important`, you eventually need another `!important` to override that one, and the cascade rapidly becomes unmanageable.

In the example above, the ID selector (`#special-message { color: green; }`) wins, regardless of the fact that it's written first in the file — **specificity, not source order, decides the winner when selectors have different specificity**. Source order only matters as a **tiebreaker**, when two rules have exactly equal specificity — in that case, whichever rule appears later in the CSS wins:

```
p { color: black; }
p { color: purple; } /* this one wins -- same specificity as above, but written LATER */
```

The box model — the single most important idea in all of CSS

Every single element on a web page is rendered as a rectangular box, whether it's a paragraph, a heading, an image, or an entire section — and every one of those boxes is built from exactly four layers, nested inside each other like rings:



Each of the four layers can be controlled individually per side, or all at once:

```

.box {
  margin-top: 10px;
  margin-right: 20px;
  margin-bottom: 10px;
  margin-left: 20px;

  /* shorthand for the exact same thing, in TOP - RIGHT - BOTTOM - LEFT order (clockwise, starting top) */
  margin: 10px 20px 10px 20px;

  /* if you only give two values, the first is used for top/bottom, the second for left/right */
  margin: 10px 20px;

  /* one value applies to all four sides equally */
  margin: 10px;
}

```

`padding` works identically to `margin`, just for the inner layer instead of the outer one. `border` needs three pieces of information (width, style, and color): `border: 2px solid black;`.

The genuinely crucial gotcha: what does `width: 200px` actually measure?

By default, in CSS's traditional box model, `width` and `height` only measure the **content** area — padding and border then get added *on top of* that, making the element's actual, total rendered size larger than the `width` you specified:

```

.box {
  width: 200px;
  padding: 20px;
  border: 2px solid black;
}
/* actual total rendered width = 200 (content) + 20 + 20 (left+right padding) + 2 + 2 (left+right border)

```

This trips up nearly every beginner at least once — you set `width: 200px`, add some padding to give the content breathing room, and suddenly the box is wider than you expected and doesn't line up the way you planned. The fix, universally applied in essentially every modern real-world project, is one single declaration:

```

* {
  box-sizing: border-box;
}

```

`box-sizing: border-box` changes what `width` measures: it now means "this box's *total* width, padding and border included" — so if you set `width: 200px` with `box-sizing: border-box`, the box is genuinely, exactly 200px wide overall, no matter how much padding or border you add (the browser shrinks the available content area to compensate, instead of growing the total box). The `*` selector matches literally every single element on the page, which is why this one rule, placed once near the top of your stylesheet, fixes this behavior everywhere at once. **You'll add this exact rule to the very top of every stylesheet you write for the rest of this course, and almost certainly for the rest of your career** — it's such a universal, sensible default that its *absence*, not its presence, is what would need explaining in a real project.

Exercises

Open `starter.html` and `starter.css` together, follow the `<!-- TODO -->` `/* TODO */` markers in both files, and check your work by reloading in the browser and comparing against `CHECKLIST.md`.

Day 4 - CSS Layout Fundamentals: Display, Position, Margin Collapsing

Day 4 — CSS Layout Fundamentals: Display, Position, and Margin Collapsing

Objectives

- Understand the `display` property and the difference between block, inline, and inline-block elements
- Understand `position` and the different "positioning contexts" it creates
- Understand a classic beginner-confusing behavior: margin collapsing
- Build enough of a foundation here that Flexbox (Day 5) and Grid (Day 6) will make immediate sense as *solutions* to problems this day's tools can't solve well

`display` — how an element behaves in the normal flow of the page

Every HTML element has a default `display` value that determines two things: whether it starts on its own new line, and whether you're allowed to set a `width/height` on it directly. The three values you'll use constantly:

block — takes up the *entire width available* to it, and always starts on a brand new line, pushing anything after it down below. You CAN set `width/height` on a block element directly. Elements like `<div>`, `<p>`, `<h1>`-`<h6>`, `<ul>`, `<li>` are **block** by default.

```
div { display: block; } /* this is already the default for div -- shown explicitly for clarity */
```

inline — takes up *only as much width as its content needs*, sits inline with surrounding text (doesn't force a new line), and — this is the part that trips people up — **setting `width` or `height` on an inline element has no effect at all**; the browser simply ignores it. Elements like `<a>`, `<span>`, `<strong>`, `<em>` are **inline** by default.

```
<p>This is <span style="width: 500px;">some text</span> in a paragraph.</p>
<!-- the width: 500px above is silently IGNORED, because span is inline by default -->
```

inline-block — the useful hybrid: behaves like **inline** in that it doesn't force a new line and sits alongside other content, but, unlike plain **inline**, it DOES respect `width/height` that you set on it. This is genuinely useful whenever you want several items sitting next to each other, each with a controlled size — though, as you'll discover on Day 5, Flexbox is usually a cleaner, more powerful tool for that specific goal, and **inline-block** is used less often in modern code as a result. It's still worth understanding, since you'll encounter it in existing codebases.

A fourth value worth knowing: **none** — completely removes the element from the page, as if it didn't exist at all (it takes up no space whatsoever, unlike merely making something invisible, which you'll meet with the separate `visibility` property later in this course).

```
.hidden { display: none; }
```

`position` — taking an element out of (or adjusting it within) the normal flow

By default, every element has `position: static` — meaning it simply sits in the normal, top-to-bottom, left-to-right document flow, exactly where its HTML places it, unaffected by any `top/left/right/bottom` properties (which only work once you change `position` away from `static`). Four other values change this behavior fundamentally:

relative — the element still takes up its normal spot in the flow (nothing else moves to compensate), but you can now nudge it visually using `top/left/right/bottom`, offset from where it would otherwise have sat:

```
.nugged {
  position: relative;
  top: 10px;    /* moves it 10px DOWN from its normal position */
  left: 20px;   /* moves it 20px RIGHT from its normal position */
}
```

Note this only shifts the *visual* rendering — the space the element originally occupied in the flow is still reserved, unaffected, which can create visual overlap with surrounding elements. `position: relative` has one more critical job, described next.

absolute — completely removes the element from the normal flow (surrounding elements behave as if it isn't there at all, collapsing into the space it would have taken), and positions it using `top/left/right/bottom` measured relative to its **nearest positioned ancestor** — meaning the closest parent element (however many levels up) that has a `position` value other than `static`. If no ancestor at all has a non-static position, it positions relative to the entire page instead.

```
.container {
  position: relative;    /* this becomes the "positioned ancestor" for the child below */
}
.badge {
  position: absolute;
  top: 5px;
  right: 5px;    /* pins this element to the top-right CORNER of .container, not the whole page */
}
```

This is precisely *why* you'll often see a seemingly-pointless `position: relative;` (with no `top/left` values at all) on a parent element — its entire purpose is purely to establish that parent as the reference point ("positioning context") for an absolutely-positioned child inside it. Without it, that child would instead position itself relative to the entire page — rarely what you actually want.

fixed — also removed from the normal flow, but positioned relative to the browser window itself, and stays anchored to that exact spot on screen even as the user scrolls the page. Common use: a navigation bar that stays visible at the top of the screen no matter how far down the page you've scrolled.

sticky — a hybrid: behaves like **relative** (in the normal flow) until the page scrolls to the point where it would go offscreen, at which point it "sticks" and behaves like **fixed** for as long as its containing parent is still on screen. A very common real use: a section heading that stays pinned to the top while you scroll through that section's content, then scrolls away normally once you reach the next section.

Margin collapsing — a classic "why is there extra space here" gotcha

Here's a genuinely surprising behavior that confuses nearly every beginner at least once: when two **block** elements are stacked vertically, and the first one has a **margin-bottom**, and the second has a **margin-top**, **the two margins do not simply add together — the browser collapses them into a single margin, equal to whichever of the two values was larger** (not their sum):

```
.first { margin-bottom: 30px; }  
.second { margin-top: 20px; }  
  
<div class="first">First box</div>  
<div class="second">Second box</div>
```

You might expect the visible gap between these two boxes to be **30px + 20px = 50px**. It's actually just **30px** — the larger of the two, not the sum. This is called **margin collapsing**, and it only happens for *vertical* margins between block-level elements in the normal document flow (it does NOT happen for padding, for horizontal margins, or for elements inside a Flexbox/Grid container, which you'll meet the next two days — this is one of several small, real reasons Flexbox and Grid feel more predictable to work with once you get there). Knowing this rule exists — even before you've fully internalized every edge case of exactly when it applies — is what turns "why is this spacing wrong, this is so confusing" into "oh, that's margin collapsing" the moment you see it happen in your own work.

Exercises

Open **starter.html** and **starter.css**, follow the `<!-- TODO -->` `/* TODO */` markers, and check your work by reloading in the browser and comparing against **CHECKLIST.md**.

Day 5 - Flexbox

Day 5 — Flexbox

Objectives

- Build the core mental model: a flex container and its flex items, and the main axis vs. cross axis
- Use `justify-content` and `align-items` to control alignment along each axis
- Use `flex-grow`, `flex-shrink`, and `flex-basis` to control how items resize
- Build several genuinely common real layouts (navbar, centering, equal-height columns) using only Flexbox

Why Flexbox exists

Yesterday's tools (`display`, `position`) can position individual elements, but they're genuinely painful for a very common need: **arranging a row (or column) of items, evenly spaced or aligned, that resize sensibly as the available space changes**. Before Flexbox existed, developers relied on hacky workarounds (misusing `float`, or the table-for-layout anti-pattern from Day 2) to achieve what should be simple layouts. **Flexbox** ("Flexible Box Layout") was introduced specifically to solve this class of problem well, and it's used constantly in real, modern websites.

The core mental model: container and items

Flexbox always involves exactly two roles: a **flex container** (the parent, where you turn Flexbox on) and its **flex items** (its direct children, which then automatically follow Flexbox's layout rules). You turn an element into a flex container with one declaration:

```
.container {  
  display: flex;  
}
```

The instant you do this, every *direct child* of `.container` automatically becomes a flex item and starts following Flexbox's layout rules — no changes needed on the children themselves for this basic behavior. The single most immediately visible effect: **flex items line up in a row, side by side, automatically** — no more `display: inline-block` or float hacks needed just to get things sitting next to each other.

Main axis vs. cross axis — the idea that unlocks everything else

This is the concept that makes every other Flexbox property make sense, so read it carefully. A flex container has two perpendicular axes:

- The **main axis** — the primary direction items are laid out along. By default, this is **horizontal** (left to right).
- The **cross axis** — perpendicular to the main axis. By default, this is **vertical**.

```
.container {
  display: flex;
  flex-direction: row;      /* the DEFAULT -- main axis is horizontal, items go left to right */
  /* flex-direction: column; -- this would make the main axis VERTICAL instead, items stack top to bottom */
}
```

Every alignment property in Flexbox is defined *relative to whichever axis is currently the main axis* — this is exactly why **flex-direction** is the very first thing to understand, since it determines what "main axis" and "cross axis" actually mean for the rest of your rules.

Aligning along the main axis: `justify-content`

```
.container {
  display: flex;
  justify-content: flex-start; /* DEFAULT -- items packed at the start of the main axis */
  justify-content: flex-end;  /* items packed at the end */
  justify-content: center;    /* items packed in the center */
  justify-content: space-between; /* first item at start, last at end, even gaps between the items */
  justify-content: space-around; /* even gaps around EACH item (including half-gaps at both ends) */
  justify-content: space-evenly; /* perfectly even gaps everywhere, including both ends */
}
```

space-between is, in practice, the single most common choice for things like a navigation bar with a logo on the left and links on the right — it's worth trying all five values against the same row of items in today's exercise so the differences are visually obvious rather than abstract.

Aligning along the cross axis: `align-items`

```
.container {
  display: flex;
  align-items: stretch; /* DEFAULT -- items stretch to fill the container's cross-axis size */
  align-items: flex-start; /* items align to the start of the cross axis */
  align-items: flex-end; /* items align to the end of the cross axis */
  align-items: center; /* items centered on the cross axis */
}
```

This is, genuinely, the most common way to vertically center something in CSS — a task that was notoriously awkward before Flexbox existed: **display: flex;** plus **align-items: center;** on the parent centers its children vertically with almost no extra effort. **align-self** (set on an individual flex item, not the container) lets you override **align-items** for just that one specific item, when everything else should align one way but a single item needs to be different.

Controlling how items grow and shrink: `flex-grow`, `flex-shrink`, `flex-basis`

By default, flex items are only as big as their content requires, and any leftover space in the container is simply left empty. Often, you want items to expand and fill that leftover space instead:

```
.sidebar {
  flex-grow: 0; /* DEFAULT -- don't grow to fill extra space */
}
.main-content {
  flex-grow: 1; /* grow to fill the container's leftover space */
}
```

```

        flex-grow: 1;      /* DO grow to consume any leftover space in the container */
    }

```

If several items all have `flex-grow: 1`, the leftover space is split evenly among them; if one item has `flex-grow: 2` and others have `flex-grow: 1`, that item gets twice as large a share of the leftover space as each of the others — `flex-grow` values describe a *ratio*, not an absolute size.

`flex-shrink` works in reverse: it controls whether an item is allowed to shrink *below* its natural size when the container is too small to fit everything (default `1`, meaning "yes, allowed to shrink" — set to `0` to prevent an item from ever shrinking, even if it causes overflow). `flex-basis` sets an item's initial, "ideal" size along the main axis, before any growing or shrinking is applied — often used instead of `width` for flex items, since it's specifically main-axis-aware.

The shorthand you'll see constantly in real code combines all three: `flex: 1`; is shorthand for `flex-grow: 1; flex-shrink: 1; flex-basis: 0%;` — an extremely common pattern meaning "this item should grow to fill available space, ignoring its content's natural size as a starting point."

`gap` — spacing between flex items, the modern easy way

```

.container {
    display: flex;
    gap: 20px;      /* 20px of space between EVERY adjacent pair of flex items, but not around the outer edges */
}

```

`gap` is dramatically simpler than the older approach of adding `margin` to individual items (which requires careful handling of the first/last item to avoid unwanted extra space at the very edges) — always reach for `gap` on a flex or grid container instead.

Three real layouts, built entirely with what you now know

A navbar (logo left, links right, vertically centered):

```

.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
}

```

Perfect centering (one item, centered both horizontally and vertically, in its parent):

```

.centered-container {
    display: flex;
    justify-content: center;
    align-items: center;
    height: 300px;
}

```

Equal-height columns (a notoriously awkward problem before Flexbox — with `align-items: stretch`, the default, every column automatically matches the height of the tallest one, with zero extra code):

```

.columns {
    display: flex;
    gap: 20px;
    /* align-items: stretch is the default -- no need to write it explicitly */
}

```

```
}  
.column {  
  flex: 1;    /* each column grows equally to fill the available width */  
}
```

Exercises

Open `starter.html` and `starter.css`, follow the `<!-- TODO --> /* TODO */` markers, and check your work against `CHECKLIST.md`.

Day 6 - CSS Grid

Day 6 — CSS Grid

Objectives

- Build the core mental model: a grid container defining rows AND columns simultaneously, unlike Flexbox's single axis
- Use `grid-template-columns/grid-template-rows` and the `fr` unit
- Place items precisely using named grid lines and `grid-area`
- Know, clearly and confidently, when to reach for Grid instead of Flexbox

Why Grid exists, given that Flexbox already exists

Yesterday's Flexbox is fundamentally **one-dimensional** — at any given moment, you're arranging items along a single main axis (a row, or a column), even though items can visually wrap onto multiple lines. **CSS Grid is two-dimensional** — you define rows and columns **together**, as one coordinated structure, and can place items into specific cells of that structure precisely. This makes Grid the better tool for genuine page-level or section-level layouts (an overall page structure with a header, sidebar, main content, and footer, for instance), while Flexbox tends to shine at arranging a row or column of related items (a navbar, a button group, a list of cards) — you'll get a clear, direct answer to "which one should I use" at the end of today's lesson.

The core mental model: defining a grid

```
.container {  
  display: grid;  
  grid-template-columns: 200px 200px 200px;    /* THREE columns, each exactly 200px wide */  
  grid-template-rows: 100px 100px;             /* TWO rows, each exactly 100px tall */  
}
```

The instant you set `display: grid`, the direct children of `.container` automatically become grid items and, by default, fill the grid's cells in order — left to right, then wrapping to the next row — very similar to how text fills a page.

The `fr` unit — Grid's most distinctive, useful tool

Rather than only specifying fixed pixel widths, Grid introduces a special unit, `fr` (short for "fraction"), representing a share of whatever space remains after any fixed-size columns/rows are accounted for:

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;    /* THREE equal-width columns, each taking an equal 1/3 share of space */  
}  
  
.container {
```

```

    display: grid;
    grid-template-columns: 200px 1fr;    /* a FIXED 200px sidebar, then the remaining space fills the rest */
}

```

This second example — one fixed-width column plus one `1fr` column — is an extremely common real layout: a sidebar with a fixed, predictable width, and a main content area that automatically fills whatever space remains, however wide or narrow the browser window happens to be. You can mix as many fixed and `fr` values as you like: `grid-template-columns: 150px 1fr 2fr;` gives a fixed 150px column, then splits the *remaining* space into three shares, giving one share to the second column and two shares (twice as much space) to the third.

`gap` — identical to Flexbox

```

.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 20px;    /* space between EVERY row and column gap, simultaneously */
}

```

You can also control row and column gaps independently with `row-gap` and `column-gap`, if you need them to differ.

Placing items precisely with grid lines

Grid automatically creates numbered **grid lines** at the edges of every row and column — for a 3-column grid, there are 4 vertical grid lines (numbered 1 through 4: before column 1, between 1 and 2, between 2 and 3, and after column 3). You can explicitly place any item to span specific lines, completely overriding the automatic left-to-right filling behavior:

```

.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);    /* shorthand for 1fr 1fr 1fr -- "repeat this pattern 3 times" */
  gap: 20px;
}
.featured-item {
  grid-column: 1 / 3;    /* span from grid line 1 to grid line 3 -- i.e., occupy the first TWO columns */
}

```

`repeat(3, 1fr)` is worth calling out on its own: it's shorthand meaning "repeat this pattern 3 times," and is far more common in real code than manually writing out `1fr 1fr 1fr` — you'll use `repeat(...)` constantly once you're building anything with more than 2-3 columns.

`grid-template-areas` — naming regions of your layout directly

This is, genuinely, one of CSS's most beginner-friendly, readable features, because it lets your CSS visually *look like* the layout it produces:

```

.page {
  display: grid;
  grid-template-columns: 200px 1fr;
}

```

```

    grid-template-rows: auto 1fr auto;
    grid-template-areas:
      "header header"
      "sidebar main"
      "footer footer";
    gap: 15px;
    min-height: 100vh;
  }
  .header { grid-area: header; }
  .sidebar { grid-area: sidebar; }
  .main { grid-area: main; }
  .footer { grid-area: footer; }

```

Each quoted line in `grid-template-areas` represents one entire row of the grid, and each word represents one cell in that row — repeating the same name across multiple cells (like `header` appearing twice in the first row) makes that one named area span all of those cells. Then, each individual child element just declares `grid-area: header`; (or whichever name applies) to be placed into that named region — no manual line-number counting required at all, and the CSS itself visually resembles a rough sketch of the actual page layout, which makes it easy to read back later and immediately understand the overall structure.

Grid vs. Flexbox — the actual decision, answered clearly

This is a genuinely common, entirely reasonable beginner question, and it deserves a direct, confident answer rather than "it depends" hand-waving:

- **Reach for Grid** when you're laying out a genuine two-dimensional structure — rows AND columns need to be coordinated together, such as an overall page layout (header/sidebar/main/footer), a photo gallery, or a dashboard of cards arranged in a grid.
- **Reach for Flexbox** when you're arranging a single row OR a single column of related items, where the *other* dimension doesn't really need independent control — a navbar, a button group, a list of tags, centering one thing inside another.
- **They are frequently used together, at different levels of the same page** — a very common, entirely normal real-world pattern is Grid for the page's overall structure, with Flexbox used *inside* one of those grid areas (say, inside the header) to arrange its own internal row of items. Neither tool "replaces" the other; they solve different shaped problems, and most real, modern websites use both.

Exercises

Open `starter.html` and `starter.css`, follow the `<!-- TODO -->` `/* TODO */` markers, and check your work against `CHECKLIST.md`.

Day 7 - Mini Project: Portfolio/Landing Page (Week 1 Review)

Day 7 — Mini Project: Personal Portfolio/Landing Page (Week 1 Review)

Objectives

Combine everything from Days 1-6 into one real, complete page:

- Document structure and semantic HTML5 elements (Days 1-2)
- Forms (Day 2)
- Selectors, the cascade, specificity, the box model (Day 3)
- Display, position, margin (Day 4)
- Flexbox (Day 5)
- Grid (Day 6)

This is intentionally less hand-held than the previous days — the point of a review project is to make the design decisions yourself, using what you've learned, and get stuck in the productive way.

The brief

Build a single-page personal portfolio/landing page, `index.html` plus `styles.css`, for a fictional (or real, if you like) person. It should include:

- 1 A **header** with a name/logo and a navigation bar (Flexbox: logo left, links right)
- 2 A **hero section** — a large introductory area with a heading, a short tagline, and maybe a call-to-action button, vertically and horizontally centered (Flexbox)
- 3 A **projects/work section** — at least 3 "cards" (each with a title and a short description) arranged in a responsive grid (CSS Grid, using `repeat()` and `fr`)
- 4 An **about section** — a paragraph or two about the person, using proper semantic elements
- 5 A **contact section** — a real form (name, email, message, submit button), with every input properly connected to a `<label>`
- 6 A **footer** with a copyright line

Suggested (not mandatory) structure

```
day07-mini-project/  
■■■ index.html  
■■■ styles.css
```

One HTML file, one CSS file, linked together — you don't need multiple pages yet (that comes with the Day 14 capstone).

Things to deliberately think about, not just copy from earlier days

- Where does `box-sizing: border-box` go, and why should it be one of the very first rules in your stylesheet? (Day 3)
- Which sections should be Flexbox, and which should be Grid? Reason through it using Day 6's "when to use which" guidance, rather than defaulting to whichever you remember better.
- Are you using semantically correct elements (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`) instead of an unbroken wall of generic `<div>`s? (Days 1-2)
- Is every form input properly connected to a `<label>` via matching `for/id`? (Day 2)
- Does your CSS have any selector conflicts you didn't intend? Use DevTools to check which rule is actually winning on any element that doesn't look right (Day 3's specificity).

How to approach this (process, not just code)

- 1 Write the full HTML structure FIRST, with no CSS linked at all, and confirm it makes sense and reads correctly even totally unstyled (this is exactly the "semantic HTML describes what things ARE" principle from Day 1-2 — a good structure should be sensible even before any visual design is applied).
- 2 Link an empty `styles.css` and add `box-sizing: border-box` as your very first rule.
- 3 Style one section at a time, top to bottom, checking in the browser after each one rather than writing all your CSS blind and debugging everything at once at the end.
- 4 Once every section looks right individually, check the whole page together, and adjust spacing (margin/padding/gap) so it reads as one coherent page, not several disconnected sections.

What's provided

- `starter.html` and `starter-styles.css` — a minimal, empty scaffold (just the file structure, no actual content or styling) to get you started, optional to use.
- `solution/` — a full reference implementation. **Build your own first.** This project has many equally valid designs; comparing your approach to this one afterward is valuable, but only after you've made your own decisions and hit your own bugs.

Checklist

See `CHECKLIST.md` in this folder for the full manual verification list once you're done.

Day 8 - Responsive Design: Media Queries, Units, Mobile-First

Day 8 — Responsive Design: Media Queries, Units, Mobile-First

Objectives

- Understand what "responsive" actually means and why it's non-negotiable for any real website today
- Use media queries to apply different CSS at different screen widths
- Understand the difference between mobile-first and desktop-first, and why mobile-first is the modern default
- Choose the right unit for the job: px, %, em, rem, vw, vh
- Understand the viewport meta tag and why every page needs it

What does "responsive" actually mean?

A **responsive** web page is one that adapts its layout to look good and remain usable across a wide range of screen sizes — a huge desktop monitor, a laptop, a tablet, and a phone, all viewing the exact same page, all getting a layout appropriate to their screen. This isn't optional polish anymore — a large share of all web traffic, for most sites, comes from phones, and a page that only works well on a wide desktop screen is, in a very real sense, broken for a huge portion of potential visitors.

The viewport meta tag — required on every page, without exception

```
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
```

Without this one line, mobile browsers assume your page was designed for a wide desktop screen and automatically zoom out to fit that assumed-wide layout onto the phone's small screen, rendering everything tiny and requiring the visitor to manually pinch-zoom just to read anything. This meta tag tells the browser: "the width of this page should match the actual width of the device's screen, and start at normal (1.0) zoom" — which is what allows your responsive CSS (media queries, below) to actually take effect the way you intend. **Add this to the `<head>` of every single page you build, starting today, for the rest of this course and beyond.**

Media queries — applying CSS conditionally, based on screen width

```
.container {
```

```

    display: flex;
    flex-direction: row;
}

@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
}

```

A **media query** wraps a block of CSS rules in a condition — here, "only apply this if the browser's viewport width is 768 pixels or less." Outside that condition (on a wider screen), the original `flex-direction: row` rule from earlier in the file still applies; once the screen narrows to 768px or below, the media query's rule kicks in and overrides it, stacking the flex items vertically instead — a very common real pattern for adapting a side-by-side layout into a stacked one on a phone.

`max-width` matches "this width or narrower"; the less common `min-width` matches "this width or wider" — which one you reach for depends on which overall strategy you're using, explained next.

Mobile-first vs. desktop-first — and why mobile-first is the modern default

Desktop-first means you write your default CSS assuming a large desktop screen, then use `max-width` media queries to override and simplify the layout for progressively smaller screens. **Mobile-first** means the reverse: your default (un-wrapped) CSS assumes a small phone screen, and you use `min-width` media queries to progressively **add** complexity as the screen grows larger:

```

/* Mobile-first: this is the DEFAULT, unwrapped CSS -- assume a phone screen */
.container {
  display: flex;
  flex-direction: column;
}

/* THEN, only once the screen is at least 768px wide, switch to a row layout */
@media (min-width: 768px) {
  .container {
    flex-direction: row;
  }
}

```

Mobile-first is the modern, generally recommended default, for a couple of concrete, practical reasons: it forces you to design the simplest, most essential version of your page first (What does this page actually need, at its core, once you strip away extra decoration that only fits on a spacious screen?), and it tends to produce less CSS overall, since you're only **adding** rules for extra space as it becomes available, rather than writing a complex desktop layout and then fighting to **undo** parts of it for a phone.

A common set of **breakpoints** (the specific widths at which your layout changes) you'll see frequently in real projects, though these are conventions, not hard rules enforced by CSS itself: roughly 576px (small phones), 768px (tablets), 992px (small laptops), 1200px (large desktops) — you'll choose your own based on where your **specific** design actually starts looking cramped or awkwardly spaced out, rather than blindly copying these numbers.

Choosing the right unit for the job

- **px** (pixels) — an absolute, fixed unit. Predictable, but doesn't scale with anything else — good for things that genuinely should stay a fixed size regardless of context, like a **1px** border.
- **%** (percentage) — relative to the **parent** element's size. **width: 50%** means "half of whatever my parent's width happens to be" — genuinely useful for fluid layouts, but its meaning changes depending on what the parent is, which can get confusing in deeply nested layouts.
- **em** — relative to the **current element's** font size (or, if the element itself doesn't set one, its nearest ancestor's). **2em** means "twice the current font size." Because **em** compounds through nested elements (an element's **em** is based on its own computed font size, which might itself have been set in **ems** relative to something else), it can produce surprising, hard-to-predict results in deeply nested structures.
- **rem** ("root em") — relative to the **root** element's font size (**<html>**, which defaults to 16px in essentially every browser unless you change it) — always, no matter how deeply nested the element using it is. This predictability is exactly why **rem** is strongly preferred over **em** for most sizing in modern CSS: **1.5rem** always means "1.5 times the root font size," full stop, regardless of nesting.
- **vw/vh** ("viewport width/height") — relative to the **browser window's** current width/height. **100vw** means "the full width of the browser window," and **50vh** means "half the height of the browser window" — genuinely useful for things that should scale directly with the screen itself, like a full-screen hero section (**height: 100vh;**).

A very common, sensible modern convention: use **rem** for font sizes and most spacing (so that if a visitor increases their browser's default font size for accessibility reasons, your whole layout scales proportionally with it), **%** or **fr**/Grid-Flexbox tools for fluid container widths, and **px** only for things that genuinely should never scale, like a thin border.

Testing responsiveness with DevTools

Every browser's DevTools includes a **device toolbar** (usually a small phone/tablet icon in the DevTools toolbar, or Ctrl+Shift+M) that lets you preview your page at different screen widths — including specific real device presets (iPhone, iPad, and others) — without needing an actual physical phone to test on. Get in the habit of opening this and dragging the width slider (or picking device presets) every time you build something responsive, checking that your media query breakpoints actually kick in where you expect and that nothing looks broken or overlapping at any width in between.

Exercises

Open **starter.html** and **starter.css**, follow the **<!-- TODO --> /* TODO */** markers, and check your work using DevTools' device toolbar, comparing against **CHECKLIST.md**.

Day 9 - CSS Advanced: Pseudo-classes, Transitions, Custom Properties

Day 9 — CSS Advanced: Pseudo-classes, Pseudo-elements, Transitions, Animations, Custom Properties

Objectives

- Style elements based on state and position using pseudo-classes
- Insert generated content using pseudo-elements
- Animate smoothly between states with `transition`
- Build a full animation sequence with `@keyframes`
- Define and reuse your own values with CSS custom properties (variables)

Pseudo-classes — styling based on state or position, without extra HTML or JavaScript

A **pseudo-class** selects an element based on some **state** or **position** it's currently in, rather than its tag, class, or id — written with a single colon:

```
a:hover {
  color: red;    /* while the mouse is hovering over a link */
}
input:focus {
  border-color: blue;    /* while an input currently has keyboard focus (e.g. just clicked into) */
}
li:first-child {
  font-weight: bold;    /* only the FIRST li among its siblings */
}
li:last-child {
  border-bottom: none;    /* only the LAST li among its siblings */
}
li:nth-child(2) {
  color: gray;    /* only the SECOND li among its siblings, counting from 1 */
}
tr:nth-child(odd) {
  background-color: #f4f4f4;    /* every odd-numbered row -- a classic "zebra striping" pattern */
}
```

`:hover` and `:focus` are especially important for a reason beyond mere visual polish: they're how you give a visitor **feedback** that something is interactive and currently being pointed at or interacted with — a link that visually changes on hover signals "this is clickable" before the visitor even clicks, and a clearly visible `:focus` style is genuinely important for anyone navigating your page with a keyboard instead of a mouse (you'll revisit this specific accessibility point on Day 10). **Never remove focus styling (`outline: none;`) without**

providing an equally visible replacement — doing so makes a page unusable for keyboard-only visitors, who rely entirely on that visual indicator to know where they currently are on the page.

`:nth-child()` accepts more than just plain numbers: `:nth-child(odd)`/`:nth-child(even)` (alternating rows), `:nth-child(3n)` (every third element), and others — worth knowing these exist, even if you only reach for the simple cases most often.

Pseudo-elements — targeting a *part* of an element, or inserting content that isn't in your HTML at all

A **pseudo-element** is written with a double colon (`::`) and lets you style a specific *part* of an element, or insert generated content that doesn't exist anywhere in your actual HTML:

```
p::first-line {
  font-weight: bold;    /* styles only the first LINE of text, wherever it happens to wrap */
}
.quote::before {
  content: "\201C";      /* inserts an opening curly quote mark BEFORE the element's actual content */
}
.quote::after {
  content: "\201D";      /* inserts a closing curly quote mark AFTER the element's actual content */
}
```

`::before` and `::after` require a `content` property to actually display anything (even `content: ""`; for an empty, purely decorative box) and are extremely common in real CSS for small decorative touches — a bullet icon, a decorative quote mark, a small badge — without needing to add an extra, purely-decorative element into your actual HTML just to hold it.

`transition` — smoothly animating between two states

Without any transition, a CSS property that changes (say, on `:hover`) snaps instantly from its old value to its new one. `transition` tells the browser to smoothly interpolate between the two values over a specified duration instead:

```
.button {
  background-color: steelblue;
  transition: background-color 0.3s ease;
}
.button:hover {
  background-color: darkblue;
}
```

Read the `transition` declaration as: "for the `background-color` property specifically, when it changes, animate smoothly over `0.3s` (300 milliseconds), using an `ease` timing curve (starts slightly slow, speeds up, slows down again near the end — the most natural-feeling default for most UI animations)." You can transition multiple properties at once, either by listing several comma-separated, or using `transition: all 0.3s ease`; to apply the same timing to every property that changes — the latter is convenient but slightly less precise, since it applies to literally every animatable property, whether you intended that or not.

This single technique — hover/focus state change plus a `transition` — is responsible for a huge share of what makes a modern website feel "polished" versus feeling static and abrupt; it's a small addition with a

disproportionately large effect on how professional an interface feels.

`@keyframes` — defining a full, multi-step animation sequence

`transition` only animates between exactly two states (the "before" and "after"). For a more elaborate sequence — several steps, or something that repeats automatically without needing a trigger like `:hover` — use `@keyframes`:

```
@keyframes bounce {
  0% { transform: translateY(0); }
  50% { transform: translateY(-20px); }
  100% { transform: translateY(0); }
}

.bouncing-ball {
  animation: bounce 1s ease-in-out infinite;
}
```

`@keyframes bounce { ... }` defines the named sequence itself — a set of percentage "checkpoints" through the animation's timeline (you can use as many as you like, not just 0/50/100), each describing what the element's styles should be at that exact point. `animation: bounce 1s ease-in-out infinite;` then applies that named sequence to an element: run the `bounce` sequence, taking `1s` total, with an `ease-in-out` timing curve, repeating `infinite` (forever) — other common values instead of `infinite` include a specific number like `3` (run it three times then stop).

`transform` (used above) is worth knowing about even briefly: it lets you move (`translateX/translateY`), resize (`scale`), or rotate (`rotate`) an element visually, without affecting the normal document flow at all (surrounding elements don't shift to compensate) — it's the property you'll reach for constantly alongside both `transition` and `@keyframes`.

CSS custom properties (variables) — define a value once, reuse it everywhere

```
:root {
  --primary-color: steelblue;
  --spacing-unit: 8px;
}

.button {
  background-color: var(--primary-color);
  padding: var(--spacing-unit) calc(var(--spacing-unit) * 2);
}

.link {
  color: var(--primary-color);
}
```

A **custom property** (informally called a "CSS variable") is declared with two leading dashes (`--primary-color`) and read back with `var(--primary-color)`. `:root` is a special selector matching the very top of the document (effectively the same as `html`, but conventionally used specifically for defining

custom properties) — declaring your variables there makes them available for use absolutely anywhere else in your stylesheet.

Why bother, instead of just retyping `steelblue` everywhere you need it? The moment you need to change your site's primary color — a genuinely common, ordinary request — you change it in exactly **one place** (the `:root` declaration), and every single rule referencing `var(--primary-color)` throughout your entire stylesheet updates automatically and consistently. Without custom properties, you'd need to hunt down and manually change every individual occurrence of `steelblue` yourself, risking missing one and ending up with an inconsistent design. This becomes dramatically more valuable as a stylesheet grows past a handful of rules — which is exactly the direction your own projects are headed as this course continues.

Exercises

Open `starter.html` and `starter.css`, follow the `<!-- TODO --> /* TODO */` markers, and check your work against `CHECKLIST.md`.

Day 10 - Typography, Color, and Accessibility

Day 10 — Typography, Color, and Accessibility

Objectives

- Choose and apply fonts properly, including a sensible fallback strategy
- Control readability with `line-height` and `letter-spacing`
- Understand the three common color formats and when each is useful
- Understand color contrast and why it's a real, measurable accessibility requirement, not a vague suggestion
- Tie together everything you've learned about `alt` text, labels, and focus styles into a coherent picture of what "accessible" actually means

Fonts and fallback stacks

```
body {  
  font-family: "Helvetica Neue", Arial, sans-serif;  
}
```

`font-family` accepts a **comma-separated list**, tried in order: the browser attempts to use the first font; if that specific font isn't installed on the visitor's device, it falls back to the next one in the list; and so on. The very last entry should always be a **generic family name** (`sans-serif`, `serif`, or `monospace`) — this guarantees that even if none of your specifically-named fonts are available, the browser still picks *some* reasonable font of the right general category, rather than falling back to something wildly inappropriate.

- `sans-serif` — fonts without small decorative strokes ("serifs") at the ends of letters (Arial, Helvetica) — generally considered cleaner and more common for screens and UI text.
- `serif` — fonts with those small decorative strokes (Times New Roman, Georgia) — often used for long-form reading text, evoking a more traditional, print-like feel.
- `monospace` — every character takes up exactly the same width (Courier, Consolas) — used for displaying code, exactly like the code blocks throughout this very course.

Web fonts — using a font that isn't already installed on the visitor's device

The fonts above are all common ones already installed on most computers. To use something more distinctive, you load a **web font** — a font file the browser downloads as part of loading your page. The most common, beginner-friendly way to do this is via **Google Fonts** (a free, hosted library):

```
<head>  
  <link rel="preconnect" href="https://fonts.googleapis.com">  
  <link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap" rel="stylesheet">  
</head>  
  
body {  
  font-family: "Roboto", Arial, sans-serif;
```

```
}
```

Notice you still include a fallback stack after your web font — if the web font fails to load for any reason (a slow connection, the service being briefly unreachable), the page still displays reasonably rather than showing completely unstyled system fonts unexpectedly.

`line-height` and `letter-spacing` — controlling readability

```
p {  
  line-height: 1.6;      /* the vertical space between lines of text, as a MULTIPLE of the font size */  
  letter-spacing: 0.02em; /* extra horizontal space between individual characters */  
}
```

line-height: 1.6 means each line takes up 1.6 times the font size in vertical space — a genuinely important, easy-to-overlook readability factor: text with lines packed too tightly together (**line-height: 1** or lower) is measurably harder to read, especially for longer paragraphs, because your eye has a harder time tracking from the end of one line to the start of the next. A value between about 1.4 and 1.6 is a common, comfortable default for body text; headings often use a tighter value since they're usually short and don't have the same "tracking between lines" problem. **letter-spacing** is used far more sparingly — usually for small, all-caps text (like a label or button), where slightly increased spacing can improve legibility.

Color formats — hex, rgb, and hsl

```
.box {  
  color: #3366cc;          /* hex: 6 hexadecimal digits -- 2 each for red, green, blue */  
  color: rgb(51, 102, 204); /* rgb: the same color, as literal 0-255 red/green/blue values */  
  color: hsl(220, 60%, 50%); /* hsl: hue (0-360 degrees), saturation %, lightness % */  
}
```

All three describe the exact same underlying color — they're just different ways of writing it down. **Hex** is the most compact and most common in real-world CSS, but it's genuinely hard for a human to reason about what color a random hex code actually represents just by reading the digits. **hsl** (hue, saturation, lightness) is often far more intuitive to work with directly, especially when you want a **family** of related colors: keep the hue the same and just change the lightness percentage to get lighter/darker versions of the exact same color, or change the hue alone (0=red, 120=green, 240=blue, moving around a color wheel) to shift to a completely different color while keeping the same saturation/lightness "feel." Many developers reach for **hsl** specifically when they need to programmatically generate a set of related shades or when tweaking a color live in DevTools, since sliding one clearly-named number (hue, saturation, or lightness) is more predictable than guessing at hex digits.

Add transparency to any of these with an added alpha value: **rgba(51, 102, 204, 0.5)** or **hsla(220, 60%, 50%, 0.5)** (the **a** stands for "alpha," where **1** is fully opaque and **0** is fully transparent) — or, in modern CSS, you can add a fourth value directly to plain **rgb()**/**hsl()** without needing the separate **a** variants at all.

Color contrast — a real, measurable accessibility requirement

Text needs sufficient contrast against its background to be readable — not just for visually impaired visitors, but for anyone using their phone outdoors in bright sunlight, or anyone with even mild, common color vision

differences. This isn't a vague design preference; it's governed by an actual published standard, **WCAG** (Web Content Accessibility Guidelines), which specifies minimum **contrast ratios** — a mathematically-calculated measure of how different two colors are in perceived brightness. The commonly-cited baseline (WCAG AA) requires a contrast ratio of at least 4.5:1 for normal body text, and at least 3:1 for large text (headings).

You do not need to calculate this ratio by hand — browser DevTools will do it for you: in the Elements panel, click on any text element, find its `color` value in the Styles panel, and clicking the small color swatch next to it typically shows the calculated contrast ratio directly, along with whether it passes the AA (and stricter AAA) thresholds. Get in the habit of checking this for your text/background color combinations, especially anything with a light gray text on a white background — a genuinely common, easy mistake that looks fine on your own well-lit monitor but fails contrast requirements and is hard for many people to read.

Pulling it all together: what "accessible" actually means, in concrete terms

You've encountered several individual accessibility-related practices scattered across the last several days — here they are, gathered into one coherent picture, since together they form the actual, practical definition of an accessible page:

- **Meaningful `alt` text** on every informative image (Day 1) — so a screen reader user knows what an image shows.
- **Properly connected `<label for>/id`** on every form input (Day 2) — so both screen readers and simple click targeting work correctly.
- **Semantic HTML elements** (`<nav>`, `<main>`, `<button>` instead of a styled `<div>`, and correct heading levels) (Days 1-2) — so a screen reader user can navigate your page's structure directly, and so keyboard users get correct default behavior (a real `<button>` is focusable and triggerable with Enter/Space automatically; a `<div>` styled to look like a button is not, unless you do significant extra work to make it so).
- **Visible `:focus` styles**, never removed without an equally visible replacement (Day 9) — so a keyboard-only user can always see exactly where they currently are on the page.
- **Sufficient color contrast** (today) — so text is genuinely readable for the widest possible range of visitors and viewing conditions.

None of these are exotic, specialized techniques reserved for some separate "accessibility mode" of a website — they're just the ordinary, correct way of using HTML and CSS, and building this awareness in **now**, in your first two weeks, means you'll simply build things correctly from the start, rather than needing to retrofit accessibility onto a project later (which is dramatically more work than doing it correctly the first time).

Exercises

Open `starter.html` and `starter.css`, follow the `<!-- TODO --> /* TODO */` markers, and check your work against `CHECKLIST.md`.

Day 11 - CSS Architecture: BEM, Organizing Stylesheets, Resets

Day 11 — CSS Architecture: BEM, Organizing Stylesheets, Resets

Objectives

- Understand the real problem large stylesheets run into, and why a naming convention like BEM exists to solve it
- Learn and apply BEM (Block, Element, Modifier) naming
- Understand how to organize CSS into multiple files as a project grows
- Understand what a CSS reset/normalize does and why browsers need one at all

The problem: CSS has no built-in way to prevent name collisions

Everything you've styled so far in this course has lived in small, single-purpose files, so naming collisions haven't been a real problem yet. But imagine a real website with 50 different components — a navbar, several kinds of buttons, cards, forms — all sharing one growing stylesheet. Two different developers (or even the same developer, weeks apart) might both reasonably create a class called `.title`, `.header`, or `.item` for two *completely unrelated* pieces of the page — and because CSS selectors aren't scoped to any particular section of the page by default, whichever rule has higher specificity (Day 3) or comes later in the file simply wins everywhere that class is used, silently breaking the other, unrelated use of that same name. This is a genuinely common, real problem in growing projects — not a hypothetical.

BEM (Block, Element, Modifier) is a naming *convention* — not a special CSS feature, just a disciplined way of naming your classes — specifically designed to prevent exactly this problem, by making every class name describe both what component it belongs to AND what role it plays within that component, so unrelated components can never accidentally collide.

BEM naming, explained through one real example

```
<div class="card">
  
  <h3 class="card__title">Product Name</h3>
  <p class="card__description">A short description.</p>
  <button class="card__button card__button--primary">Buy Now</button>
</div>

.card { }
.card__image { }
.card__title { }
.card__description { }
.card__button { }
```

```
.card__button--primary { }
```

- **Block** — a standalone, reusable component: `card`. This is the outermost, top-level name.
- **Element** — a part *of* that block, written as `block__element` (two underscores): `card__title` means "the title, specifically as it appears inside a card" — not just any generic title anywhere on the page.
- **Modifier** — a variation of a block or element, written as `block__element--modifier` (two dashes): `card__button--primary` means "the button inside a card, specifically the primary-styled variant" (as opposed to, say, a `card__button--secondary`).

Because every single class name is prefixed with its block (`card__...`), a completely unrelated part of the page — say, a navbar with its own `.navbar__title` — can never collide with `.card__title`, even though both are conceptually "a title" in plain English. The naming itself carries the necessary context to keep them apart, with zero risk of accidental overlap, and no need to rely on deeply nested selectors (`div.card > h3`) just to disambiguate them, which would otherwise increase specificity in exactly the way Day 3 warned against.

A genuinely practical side benefit: reading `class="card__button--primary"` directly in your HTML immediately tells you, without needing to go check the CSS at all, that this element is a button, specifically belonging to a card component, specifically the primary-styled variant — the naming itself documents the relationship.

Organizing CSS into multiple files as a project grows

For a small site (like everything you've built in the last ten days), one `styles.css` file is perfectly fine. As a real project grows, it's common to split styles into multiple files, organized by responsibility, and then combine them — either by linking several `<link>` tags in order, or, more commonly in real projects, using a build tool (outside this course's scope) that merges them into one file automatically. A common, sensible split:

```
styles/
■■■ reset.css           -- the reset/normalize (see below), always loaded FIRST
■■■ variables.css       -- your :root custom properties (Day 9)
■■■ base.css            -- body, headings, links -- broad, foundational defaults
■■■ layout.css          -- header, footer, page-level grid/flex structure
■■■ components/
■   ■■■ card.css        -- one file per BEM "block," e.g. all .card__ rules together
■   ■■■ button.css
■   ■■■ navbar.css
```

The exact split matters less than the underlying principle: **group related rules together, and keep the loading order intentional** (reset and variables first, since later files depend on them; broad/general rules before narrow/specific component rules, so specificity and source-order both work in your favor rather than against you, per Day 3).

CSS resets and normalize.css — why browsers need one at all

Here's a fact that surprises many beginners: **every browser ships with its own small, default stylesheet**, applied automatically to every page, before any of your own CSS runs at all — this is why an unstyled `<h1>` already appears large and bold, and an unstyled `<ul>` already has bullet points and left padding, even though

you never wrote a single rule for either. The problem: **these built-in default styles are not perfectly identical across different browsers** — a `<button>`, for instance, might have subtly different default padding, border style, or font in Chrome versus Firefox versus Safari.

A **CSS reset** (or the gentler, more modern **normalize.css**) is a stylesheet, loaded before any of your own styles, whose entire job is to flatten out these small, inconsistent browser defaults into one predictable, consistent baseline — so that the rest of your CSS behaves the same way regardless of which browser a visitor happens to be using. A minimal, commonly-used modern reset:

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```

This single, small rule — which you've actually already been using in some form since Day 3! — removes every browser's inconsistent default margin/padding on elements like `<body>`, headings, and lists, and applies the **border-box** sizing model everywhere, giving you a clean, entirely predictable starting point to build your own, deliberate spacing on top of, rather than fighting against invisible, browser-specific defaults you never asked for. Full reset libraries (like the well-known "Eric Meyer's Reset" or "normalize.css") go considerably further, addressing many more small inconsistencies across form elements, tables, and typography — worth knowing they exist and being able to add one to a real project, even though the simple version above covers the most common, impactful cases you'll run into in this course.

Avoiding specificity wars

A **specificity war** happens when developers respond to an unwanted CSS override by adding an even more specific selector (or, worse, **!important**) on top, which the next unwanted override then has to beat with something even more specific still — an escalating, increasingly unmanageable mess, exactly as warned about on Day 3. BEM directly helps prevent this: because every class name is already precisely scoped to its own component, you rarely need more than a single class selector (`.card__button--primary { }`) to target exactly the right element — there's no need to nest selectors deeply or add IDs just to "win" against some other unrelated rule, because BEM's naming convention already prevented the collision from ever being possible in the first place.

Exercises

Open `starter.html` and `starter.css`, follow the `<!-- TODO --> /* TODO */` markers, and check your work against `CHECKLIST.md`.

Day 12 - Forms in Depth and Debugging with DevTools

Day 12 — Forms in Depth and Debugging with DevTools

Objectives

- Use HTML5's built-in form validation attributes, understanding what the browser checks automatically for free
- Style validation states (`:valid/invalid`) to give visitors clear feedback
- Style form controls consistently, working around inconsistent default browser appearances
- Build a systematic process for debugging a layout that "just isn't working," using DevTools properly rather than guessing

HTML5 built-in validation — checks the browser performs automatically

Recall from Day 2 that you built a form without worrying yet about validating what visitors typed into it. HTML5 provides several attributes that make the *browser itself* enforce basic rules, with zero extra code required:

```
<input type="text" required>
<input type="email" required>
<input type="number" min="1" max="100">
<input type="text" pattern="[A-Za-z]{3,}" title="At least 3 letters, no numbers">
<input type="text" minlength="3" maxlength="20">
```

- `required` — the form cannot be submitted at all while this field is empty; the browser automatically shows its own built-in message and focuses the field if a visitor tries to submit anyway.
- `type="email"` — the browser checks that the entered text at least loosely resembles a valid email address (contains an @ and a domain-like portion) before allowing submission.
- `min/max` (on `type="number"`) — restricts the field to a numeric range.
- `pattern` — a regular expression (a precise pattern-matching syntax, outside this course's scope to teach in full) the entered text must match; `title` provides the tooltip text shown if it doesn't match.
- `minlength/maxlength` — restricts how many characters can be typed.

This built-in validation is a genuinely useful first line of defense, but it is never sufficient on its own for anything that matters — a visitor can always disable JavaScript, use a tool to submit a request directly without ever loading your page's HTML at all, or simply have an old browser that doesn't support some of these attributes. Real applications always *also* validate again on the server, after the form is actually submitted, treating any client-side validation (browser-enforced, as covered here) purely as a nicer, faster experience for well-behaved visitors — never as an actual security or data-integrity guarantee. This course doesn't cover building that server side, but the principle is worth internalizing now, before you're ever tempted to trust client-side validation alone.

Styling validation states with `:valid` and `:invalid`

These are pseudo-classes (recall Day 9's `:hover`/`:focus`) that match an input based on whether its **current** content currently satisfies its own validation rules:

```
input:invalid {
  border-color: red;
}
input:valid {
  border-color: green;
}
```

One subtlety worth knowing: an empty **required** field is considered `:invalid` by default, which means, without extra care, every required field on your form shows as an alarming red border the instant the page loads, before a visitor has even had a chance to type anything — usually not the experience you actually want. A common, better pattern only shows the invalid styling once a visitor has actually interacted with (and left) the field:

```
input:not(:placeholder-shown):invalid {
  border-color: red;
}
```

`:not(...)` is a pseudo-class that inverts whatever selector you put inside it — `:not(:placeholder-shown)` matches an input whose placeholder text is currently NOT showing, which happens precisely once a visitor has typed **something** into it (even something that turns out to be invalid) — a reasonably good, commonly-used approximation of "the visitor has actually interacted with this field," achieved with pure CSS and no extra code.

Styling form controls consistently

Recall from Day 11 that browsers apply inconsistent default styling — this shows up especially strongly on form elements: buttons, checkboxes, and select dropdowns can look meaningfully different by default across Chrome, Firefox, and Safari. A common baseline reset for text-like inputs:

```
input[type="text"],
input[type="email"],
input[type="password"],
textarea,
select {
  font-family: inherit; /* by default, some browsers use a different font for form controls than the page */
  font-size: inherit;
  padding: 8px 12px;
  border: 1px solid #ccc;
  border-radius: 4px;
  width: 100%;
}
```

Notice the selector here: `input[type="text"]` is an **attribute selector** — it matches an `<input>` element specifically when its `type` attribute equals `"text"`, letting you target one specific kind of input without needing an extra class on every single one. `font-family: inherit;` and `font-size: inherit;` are worth calling out specifically: by default, many browsers render form controls using the **operating system's** own default UI font rather than whatever font-family you set on `<body>`, which is precisely why form elements can look visually "out of place" compared to the rest of a page unless you explicitly tell them to inherit your page's own

font choices instead.

Debugging a layout that "isn't working" — a systematic process, not guessing

Every beginner eventually hits a moment where a layout simply doesn't look the way they expected, and the instinct is often to randomly change values and reload, hoping something works. Here's a genuinely more effective, systematic process, using DevTools:

- 1 **Open DevTools and click the misbehaving element directly** (right-click → Inspect). This immediately shows you the *exact* HTML for that element, and, critically, its box model diagram.
- 2 **Check the Styles panel** for every rule currently affecting that element, in the order the browser is applying them, with strikethrough text showing rules that are being overridden by something more specific (recall Day 3's specificity) — this immediately tells you if some *other* rule you forgot about is winning unexpectedly.
- 3 **Check the "Computed" tab** (next to Styles) to see the final, actual resolved value of any property, after all overrides have been applied — genuinely useful when you're not sure which of several competing rules actually won.
- 4 **Check the box model diagram** (usually visible in the Styles or Computed panel) to see the actual rendered content/padding/border/margin sizes — this immediately reveals whether an unexpected size is due to **box-sizing** (Day 3), unexpected padding, or margin collapsing (Day 4).
- 5 **Temporarily disable rules** by unchecking them in the Styles panel (rather than editing your actual file) to isolate exactly which single rule is causing the unexpected behavior — turn things off one at a time until the problem disappears, and you've found your culprit.
- 6 **Only once you've identified the actual cause** using the steps above, go edit your real **.css** file — not before, since guessing-and-reloading without this diagnostic process wastes far more time than this systematic approach, especially as your stylesheets grow larger than what you've worked with so far in this course.

This process — inspect, read the Styles panel, check the box model, isolate by disabling rules, THEN edit — is, in practice, how real, experienced developers debug CSS every single day. It's worth deliberately practicing it in today's exercise rather than skipping straight to the solution file.

Exercises

Open **starter.html** and **starter.css**. This exercise has two parts: build the validation styling, AND deliberately debug a layout bug using the process above (see the specific instructions inside the files). Check your work against **CHECKLIST.md**.

Day 13 - Practical: Project Structure, Git Workflow, Deployment

Day 13 — Practical: Project Structure, Git Workflow, Deploying a Static Site

Objectives

- Organize a multi-page site's files and folders the conventional way
- Practice a real git workflow, applied to a site project rather than just abstractly
- Understand what a "static site" is and deploy one for real, for free, so it's viewable by anyone on the internet

Organizing a real, multi-page site's files

For everything you've built so far in this course, one HTML file and one CSS file (or a small handful) was enough. A real, multi-page site needs a predictable structure so that images, styles, and pages can all find each other correctly:

```
my-site/
├── index.html           -- the homepage; browsers automatically look for this filename by default
├── about.html
├── contact.html
├── css/
│   └── styles.css
├── images/
│   ├── logo.png
│   └── hero-photo.jpg
└── README.md
```

`index.html` is a special, conventional filename: when a visitor navigates to a folder without specifying an exact file (e.g., just `https://example.com/`, or, locally, just opening the `my-site` folder), the browser (or web server) automatically looks for and serves `index.html` from that folder by default. This is why the homepage is always named `index.html`, and why you've likely already seen this filename recommended without a full explanation before now.

Referencing files across this structure uses **relative paths** — a path written relative to the *current* file's own location, rather than a full absolute address:

```
<!-- inside about.html, at the top level -->
<link rel="stylesheet" href="css/styles.css">

<a href="index.html">Home</a>
<a href="contact.html">Contact</a>
```

`css/styles.css` means "look inside a folder called `css`, sitting right next to this HTML file, for a file called `styles.css`." Get comfortable with relative paths now — a genuinely common beginner bug is an image or stylesheet that works when you open a file directly, but breaks the moment you upload the exact same project

to a real server, purely because a path assumed a folder structure that wasn't actually preserved consistently.

A real git workflow, applied to an HTML/CSS project

The core git commands and reasoning here directly mirror the same discipline used in any programming project — the practice, not the tool, is what genuinely matters:

```
git init                                # once, at the start of the project
git add .
git commit -m "Initial commit: homepage structure and base styles"

# ... later, after adding a new page ...
git status                             # see what's changed
git diff                               # see the EXACT lines changed
git add about.html css/styles.css
git commit -m "Add About page and its styles"

git log --oneline                       # see the project's history
```

****A commit message should explain *why* a change was made, not simply restate *what* changed**** — `git diff` already shows precisely *what* changed in detail; a message like `"Fix navbar overlapping on mobile"` is far more useful to you later than a vague `"update css"`. Just as in any other kind of project, work in small, logical commits as you build each page or feature, rather than one enormous commit at the very end covering the entire two weeks of work — this gives you (and, on a real team, any collaborator) a genuinely useful history to look back through later.

Branches, exactly as in any other project: create a new branch for a meaningful chunk of work (`git checkout -b feature/add-contact-page`), commit your changes there, and merge back into your main branch once it's done and working — rather than committing every single change directly onto `main`. This habit matters just as much for a website project as for any other kind of software project, and it's worth practicing here rather than treating it as something that only applies to "real programming."

Deploying a static site — making it viewable by anyone on the internet

A **static site** is one made entirely of files that don't change based on who's requesting them or need a server to run any code — exactly what you've built throughout this entire course: plain HTML, CSS, and (if you add it later) client-side JavaScript, with no database or server-side logic involved. Static sites are the simplest kind of website to deploy, and several free services exist specifically for hosting them. **GitHub Pages** is a common, beginner-friendly choice, since it's directly built into GitHub (which you're likely already using for the git workflow above):

- 1 Push your project to a GitHub repository (create one on github.com, then `git remote add origin <your-repo-url>` and `git push -u origin main`, if you haven't already connected your local repository to one).
- 2 On GitHub, go to your repository's **Settings** tab, then find the **Pages** section.
- 3 Under "Source," choose the branch (usually `main`) and folder (usually `/ (root)`, unless your `index.html` lives in a subfolder) you want published.

- 4 GitHub builds and publishes your site at a URL in the form `https://your-username.github.io/your-repo-name/` — usually within a minute or two.

Once deployed this way, any future change follows a simple, repeatable cycle: make your edit locally, test it by opening the file directly in your browser (exactly as you've done all course), commit and push it to GitHub, and the live site updates automatically shortly afterward — no separate, manual "upload" step required beyond your normal `git push`.

Exercises

This is a hands-on, do-it-for-real day rather than an auto-graded one, using the Day 7 portfolio project (or a copy of it) as your subject:

- 1 Reorganize your Day 7 portfolio into the folder structure shown above (`index.html` at the root, a `css/` folder, an `images/` folder if you used any real images), updating every path reference to match.
- 2 Confirm the reorganized site still opens and works correctly by opening `index.html` directly in your browser.
- 3 If you have git installed: run `git init` in the project folder, then make at least two separate, meaningfully-described commits as you reorganize (one for the restructuring itself, and at least one more for some small additional change or fix you make along the way).
- 4 If you have a GitHub account: push the project to a new repository and enable GitHub Pages for it, following the steps above, then visit the live URL GitHub gives you and confirm your site actually works when loaded from the real internet, not just locally.

There's no separate `solution/` for this day — the "solution" is your own reorganized Day 7 project, genuinely deployed.

Day 14 - Capstone: Multi-Page Responsive Website

Day 14 — Capstone: Multi-Page Responsive Website

Objectives

Build a complete, small, multi-page website using nearly everything from the last 13 days, mostly unassisted:

- Semantic HTML structure and forms (Days 1-2)
- Selectors, the cascade, specificity, the box model (Day 3)
- Layout fundamentals: display, position (Day 4)
- Flexbox (Day 5) and Grid (Day 6)
- Responsive design: media queries, mobile-first (Day 8)
- Pseudo-classes, transitions, custom properties (Day 9)
- Typography, color, accessibility (Day 10)
- BEM naming and organized CSS (Day 11)
- Form validation styling (Day 12)
- Proper project structure and (if applicable) real deployment (Day 13)

This is intentionally the least hand-held day. You get a specification, not a scaffold. Expect to get stuck — that's the design, not a bug in the lesson.

The brief

Build a small, real, multi-page website — a fictional (or real) small business, portfolio, or organization's site — with at least 3 pages, sharing one consistent design:

- 1 **A homepage** (`index.html`) with a hero section and a preview of your other pages/content (e.g., 3 featured items in a Grid layout).
- 2 **An "About" or "Services" page** with real semantic content structure.
- 3 **A "Contact" page** with a properly validated form (required fields, `type="email"`, styled `:valid/:invalid` states).
- 4 **A shared navigation bar and footer**, identical across all 3+ pages, correctly linking between them using relative paths.
- 5 **Fully responsive**, tested at both a phone width (around 375px) and a desktop width (1200px+), using at least one media query breakpoint that meaningfully changes the layout (not just font sizes).

Required design elements (this is the point of the exercise — don't skip these to save time)

- One shared, external stylesheet (or an organized set of them, per Day 11) — no inline `style` attributes or `<style>` blocks scattered across pages.
- Class names following BEM (Day 11) for at least your major reusable components (navbar, cards, buttons).
- At least one use of Flexbox AND at least one use of Grid, each chosen deliberately for the layout it's actually suited to (Day 6's decision criteria) — not just because you remember one better.
- A mobile-first responsive layout (Day 8) — write your default CSS for a narrow screen, then use `min-width` media queries to enhance it for wider screens.
- At least one `:hover/:focus` transition (Day 9) somewhere that benefits from it (a button, a nav link).
- CSS custom properties (Day 9) for your primary color(s), defined once in `:root` and reused throughout.
- Meaningful `alt` text on every image, properly connected `<label>`s on every form input, and visible `:focus` styles never removed without a replacement (Day 10's accessibility checklist).
- `box-sizing: border-box` and a small reset, applied globally, as the very first rules in your stylesheet (Days 3, 11).

Suggested (not mandatory) file layout

```
day14-capstone/
  ■■■ index.html
  ■■■ about.html
  ■■■ contact.html
  ■■■ css/
  ■ ■■■ styles.css
  ■■■ images/
```

How to approach 2+ hours of mostly-unassisted work without stalling out

- 1 **Plan your pages and shared layout first**, on paper or in comments, before writing any code — which sections repeat on every page (navbar, footer), and which are unique to one page?
- 2 **Build the shared navbar and footer once**, in plain HTML with no styling, and copy that exact same markup across all 3 pages before doing anything else — get the multi-page linking working and correct first.
- 3 **Style one page fully** (probably the homepage) before moving to the next, rather than half-styling all three simultaneously.
- 4 **Build mobile-first**: write your default (unwrapped) CSS assuming a narrow screen, test it at 375px in DevTools, THEN add `min-width` media queries to enhance the layout for wider screens — resist the temptation to design for desktop first and retrofit mobile after, even though that may feel more natural if you're mentally picturing your own laptop screen while working.
- 5 **Once the happy path works**, go back through Day 10's accessibility checklist and Day 12's validation styling, and deliberately verify each item rather than assuming you did it right the first time.

Definition of done

- All 3+ pages exist, share an identical navbar/footer, and link to each other correctly.
- The site is genuinely responsive — tested by hand in DevTools at both a phone width and a desktop width, with at least one layout (not just font-size) change between them.
- The contact form validates properly and shows clear `:valid/:invalid` feedback.
- Every image has meaningful `alt` text; every form input has a properly connected `<label>`.
- Your class names follow BEM for at least the reusable components (navbar, cards, buttons).

Reference solution

[solution/](#) has a complete working implementation. Give yourself a genuine attempt — ideally the full 2 hours — before opening it. The value of this whole two weeks culminates here: if you can build this mostly on your own, you're at the level this track promised.